

Modèles prédictifs et analyse de données



Dr Khadidja Henni

Séance 4: Pré-traitement des données

Plan de la séance

- Retour sur les généralités d'apprentissage machine
- Les données
- Ingénierie des caractéristiques
- Prétraitement de données
- Extraction des caractéristiques
- Laboratoire formatif

Prétraitement de données

Les techniques de prétraitement des données

- Liste des techniques de préparation des caractéristiques:
 1. Imputation (Complétion de données manquantes)
 2. Gestion des valeurs aberrantes (Outliers)
 3. Binning (Discrétisation)
 4. Transformation logarithmique (Log Transform)
 5. Encodage one-hot (One-Hot Encoding)
 6. Opérations de regroupement (Grouping Operations)
 7. Séparation des caractéristiques (Feature Split)
 8. Mise à l'échelle (Scaling)
 9. Extraction de dates (Extracting Date)

Gestion des doublons

Gestion des doublons

Un **doublon** est une ligne **identique ou très similaire** à une autre dans un jeu de données. Il peut être :

- **Exact** : toutes les colonnes sont identiques.
- **Partiel** : certaines colonnes sont identiques (ex. : même nom/prénom, mais date différente).
- **Légitime** : la duplication est réelle et justifiée (ex. : deux commandes d'un même client).

Un doublon peut donc être une **erreur** ou une **répétition valide**, d'où l'importance de l'analyse contextuelle.

D'OÙ VIENNENT LES DOUBLONS ?

Les doublons proviennent généralement de **problèmes lors de la création ou fusion des données** :

Origine	Exemple concret
Fusion de fichiers	Deux fichiers clients concaténés sans filtrer les doublons
Erreurs de saisie	Saisie répétée par erreur (copier-coller, mauvais ID)
Importations multiples	Même fichier importé deux fois dans une base
Absence de clé unique	Pas d'ID client ou ID mal géré (ex. : espaces ou majuscules)
Données légitimes doublées	Client ayant plusieurs commandes identiques

Gestion des doublons

3. POURQUOI IL FAUT LES GÉRER ?

Les doublons faussent l'analyse et peuvent introduire :

- Erreurs statistiques : fausse moyenne, comptage biaisé.
- Répétitions dans les modèles : surapprentissage en ML.
- Multiplication des coûts : si les actions sont déclenchées deux fois (ex. : e-mails).
- Erreurs métiers : plusieurs factures pour la même commande, mauvaise attribution.

Ignorer les doublons peut compromettre la qualité et la fiabilité des résultats d'analyse.

COMMENT LES DÉTECTER ET LES TRAITER ?

a) Détection avec duplicated()

```
df.duplicated() # détecte les doublons complets
df.duplicated(subset=['nom', 'email']) # sur certaines colonnes
df[df.duplicated(keep=False)] # affiche toutes les lignes en doublon
```

b) Suppression avec drop_duplicates()

```
df.drop_duplicates() # supprime doublons complets, garde la 1ère
df.drop_duplicates(subset=['email'], keep='last') # personnalisation
df.drop_duplicates(keep=False) # supprime toutes les occurrences
```

c) Cas particuliers

Type de doublon	Exemple	Action recommandée
Exact	Ligne identique	Supprimer
Partiel	Même nom/prénom, champs incohérents	Vérifier manuellement
Légitime	Plusieurs achats d'un client	Ne pas supprimer
Métier (ID commun)	Même ID, champs différents	Fusionner ou garder la version à jour

Correction des types de données erronés

Correction des types de données erronés

C'EST QUOI UN TYPE DE DONNÉES ERRONÉ ?

Un **type de données erroné** signifie qu'une colonne a un **type non adapté** à son contenu réel ou à son usage prévu.

Exemples fréquents :

- Une **date stockée comme chaîne de caractères** (object au lieu de datetime64).
- Une **valeur numérique stockée comme texte** ("45" au lieu de int ou float).
- Une **valeur catégorielle** mal interprétée comme float

Ces erreurs empêchent les **calculs**, les **triaux**, les **filtres** ou les **analyses temporelles**.

D'OÙ VIENNENT LES ERREURS DE TYPAGE ?

Origine	Exemple concret
Importation automatique	pandas.read_csv() interprète mal les colonnes
Saisie manuelle	Mélange de chiffres et lettres (ex. : "45 kg")
Données mal formatées	Dates au mauvais format ("15/03/2023" ou "03-15-23")
Valeurs manquantes ou mixtes	"?", "None", "-" mélangés avec des nombres
Export d'un logiciel externe	Export Excel → CSV : tous les types deviennent object

Correction des types de données erronés

POURQUOI IL FAUT CORRIGER LES TYPES ?

Un mauvais typage peut entraîner :

- **Erreurs de calcul** : impossible de faire des moyennes sur des chaînes (object).
- **Erreurs de filtre ou de tri** : dates non reconnues → tri alphabétique incorrect.
- **Échec d'algorithmes de ML** : certains modèles n'acceptent que des int ou float.
- **Problèmes de jointures** : deux colonnes de types différents ne peuvent pas être fusionnées.

Corriger les types est **essentiel pour la qualité, la compatibilité et la validité** des traitements.

COMMENT LES CORRIGER ?

a) Identifier les types de données

```
df.dtypes          # affiche les types de chaque colonne  
df.info()          # utile pour repérer les 'object' suspects
```

b) Conversions explicites:

• Numérique:

```
df['âge'] = df['âge'].astype(int)  
df['revenu'] = pd.to_numeric(df['revenu'], errors='coerce') # gère les erreurs
```

• Date/temps :

```
df['date_achat'] = pd.to_datetime(df['date_achat'], errors='coerce', dayfirst=True)
```

• Catégoriel :

```
df['sexe'] = df['sexe'].astype('category')
```

Correction des types de données erronés

c) Respecter la signification métier

- Une **date** doit être au format datetime pour calculer des durées.
- Un **identifiant client** reste souvent object (pas un nombre destiné à des calculs).
- Une **colonne binaire** (oui/non, 0/1) peut être convertie en bool ou int.

Exemple piège :

```
df['code_postal'] = df['code_postal'].astype(str) # pour préserver les zéros initiaux
```

Problème	Origine probable	Solution recommandée
Numérique comme texte	Saisie ou import CSV	astype(int/float)
Date non reconnue	Format ambigu	pd.to_datetime()
Type incohérent avec usage	Erreur métier ou automatique	Vérification + conversion manuelle

Harmonisation des noms et des valeurs

Harmonisation des noms et des valeurs

C'EST QUOI L'HARMONISATION ?

L'harmonisation consiste à **uniformiser les noms de colonnes** et les **valeurs textuelles** dans un DataFrame pour :

- éviter les doublons invisibles ("Homme" ≠ "homme "),
- garantir la cohérence,
- préparer les données pour l'analyse ou le machine learning.

Deux aspects sont concernés :

- Les **noms de colonnes** (df.columns)
- Les **valeurs dans les cellules**, surtout celles de type object ou category

D'OÙ VIENNENT LES INCOHÉRENCES ?

Source de variation	Exemple concret
Fichiers multiples	"Nom" dans un fichier, "nom" dans l'autre
Saisie manuelle	"Femme", "femme ", " FEMME"
Fusion de bases	Colonnes avec tirets, underscores, accents
Export de logiciels	Colonnes avec noms générés automatiquement
Problèmes linguistiques	"âge" ≠ "age", "ville" ≠ "Ville"

Harmonisation des noms et des valeurs

3. POURQUOI HARMONISER ?

- Éviter les doublons de colonnes ou valeurs équivalentes.
- Faciliter les filtres, regroupements, jointures (ex. : `groupby()`, `merge()`).
- Améliorer la lisibilité du jeu de données.
- Préparer les données pour l'apprentissage automatique (ex. : `OneHotEncoding`).

Des noms incohérents ou mal formés peuvent faire **échouer une analyse ou fausser les résultats**.

4. COMMENT HARMONISER ?

a) Renommer les colonnes avec `.rename()` ou `df.columns = [...]`

```
# Exemple simple
df.rename(columns={'Nom': 'nom', 'Âge': 'age'}, inplace=True)

# Harmonisation globale
df.columns = [col.lower().replace(' ', '_') for col in df.columns]
```

b) Corriger les chaînes de caractères (valeurs)

```
# Nettoyage général d'une colonne texte
df['sexe'] = df['sexe'].str.strip().str.lower()

# Supprimer les accents, les espaces, les caractères spéciaux (via regex)
import unicodedata
import re

def clean_string(s):
    if isinstance(s, str):
        s = s.strip().lower()
        s = unicodedata.normalize('NFD', s).encode('ascii', 'ignore').decode("utf-8")
        s = re.sub(r'^a-z0-9_', '_', s)
    return s

df['ville'] = df['ville'].apply(clean_string)
```

Harmonisation des noms et des valeurs

c) Bonnes pratiques

- Préférer des noms **courts, sans accents ni majuscules**, séparés par _ :
ex. : code_postal, date_achat, montant_total
- Nettoyer les colonnes **catégorielles** avant le codage.
- Appliquer une **stratégie uniforme** à tout le dataset dès le début du projet.

Ce qu'on corrige	Exemples avant	Exemples après
Noms de colonnes	"Code Postal", "Âge"	code_postal, age
Majuscules / espaces	" Homme ", "FEMME"	homme, femme
Accents et caractères spéciaux	"Québec", "âge"	quebec, age
Valeurs catégorielles	"oui", "Oui ", "OUI"	oui

Correction des formats dégradés ou mal formés

Correction des formats dégradés ou mal formés

C'EST QUOI UN FORMAT MAL FORMÉ ?

Un **format mal formé** est une valeur textuelle (object) censée représenter un type **standard** (date, montant, pourcentage...), mais contenant :

- des **caractères parasites** (\$, %, €, etc.),
- des **formats incohérents ou mixtes** (15-03-2023, 2023/03/15, "15 mars 2023"),
- des **chaînes combinées** à parser ("12 000 \$ CAD", "3h45min", "23°C").

2. D'OÙ VIENNENT CES FORMATS DÉGRADÉS ?

Origine	Exemple concret
Exports Excel / systèmes	"12 500 \$" ou "1 000,75 €"
Saisie libre	"23 mars 2023", "mars 23, 2023"
Données multilingues	"décembre" vs "december"
Colonnes fusionnées	"23.5°C", "12h30min"
Valeurs mixtes	"Inconnu", "NaN", "0" dans une même colonne

Correction des formats dégradés ou mal formés

3. POURQUOI IL FAUT LES CORRIGER ?

- Impossible de **trier, calculer, comparer** si le type est object.
- Les **modèles d'apprentissage automatique échouent** si les valeurs ne sont pas numériques ou datées.
- Fausses analyses statistiques ou temporelles (dates mal reconnues).
- Les formats mixtes **empêchent l'imputation ou la normalisation**.

Corriger les formats garantit une **base de données fiable, normalisée et exploitable**.

4. COMMENT LES CORRIGER ?

a) Convertir les dates en format standard ISO (datetime64)

```
df['date'] = pd.to_datetime(df['date'], errors='coerce', dayfirst=True)
```

b) Nettoyer les montants et unités avec str.replace() ou regex

```
df['salaire'] = df['salaire'].str.replace('[\$,€\s]', '', regex=True).str.replace(',', '.', '')  
df['salaire'] = pd.to_numeric(df['salaire'], errors='coerce')
```

- Supprime les symboles \$, €, les espaces et les virgules
- Convertit la chaîne nettoyée en float

Correction des formats dégradés ou mal formés

c) Parser des chaînes complexes avec regex ou extract()

```
# Extraire les heures et minutes
```

```
df[['heures', 'minutes']] = df['durée'].str.extract(r'(\d+)h(\d+)min')
```

```
# Nettoyage des températures
```

```
df['temp'] = df['temp'].str.replace('°C', '').astype(float)
```

d) Utiliser des fonctions personnalisées pour les cas complexes

```
import re
def nettoyer_montant(val):
    if isinstance(val, str):
        val = re.sub(r'^\d.,-', '', val)
        val = val.replace(',', '.')
    try:
        return float(val)
    except:
        return None
```

```
df['montant'] = df['montant'].apply(nettoyer_montant)
```

Cas fréquent	Exemple	Solution
Dates incohérentes	"2023/03/15", "15 mars"	pd.to_datetime()
Montants avec symboles	"12 000 \$"	str.replace() + to_numeric()
Textes à structure connue	"2h30min", "23°C"	str.extract() avec regex
Formats trop variés	"Inconnu", "0", "--"	Nettoyage + errors='coerce'

Prétraitement numérique des données : centrage, mise à l'échelle, transformation

Prétraitement numérique des données

1. C'EST QUOI LE PRÉTRAITEMENT NUMÉRIQUE ?

Le prétraitement numérique consiste à **transformer les colonnes numériques** avant leur utilisation dans des modèles ou des analyses, en appliquant :

- un **centrage** (soustraction de la moyenne),
- une **mise à l'échelle** (normalisation, standardisation),
- une **transformation mathématique** (log, racine, puissance, etc.)

Ces opérations changent **l'unité, l'échelle ou la distribution** sans altérer l'information fondamentale.

2. POURQUOI C'EST NÉCESSAIRE ?

a) **Les variables numériques ont souvent des échelles très différentes :**

Exemples :

Revenu annuel : 20 000 \$ – 150 000 \$

Nombre d'enfants : 0 – 5

Âge : 18 – 80

Sans mise à l'échelle, les variables **ayant de grandes valeurs dominant les calculs**.

Prétraitement numérique des données

b) Problèmes concrets si on ne prétraite pas :

Problème technique	Explication
Distance biaisée	Dans KNN, la distance sera dominée par la variable avec la plus grande échelle.
Algorithmes instables	Les coefficients explosent (régression linéaire), ou convergence lente (gradient)
Mauvaise courbe d'apprentissage	Réseaux de neurones mal entraînés si les valeurs varient trop

c) Algorithmes sensibles au prétraitement

Algorithme	Sensible à l'échelle ?	Raisons
KNN	Oui	Utilise la distance Euclidienne
SVM	Oui	Dépend du produit scalaire
Régression linéaire	Oui (en pratique)	Influence des variables avec grandes valeurs
Réseaux de neurones	Oui	Gradients instables si les valeurs ne sont pas centrées
Arbres de décision	Non	Fonctionnent par découpage (pas affectés par les échelles)

Standardisation

Standardisation (centrage-réduction) :

- **Définition et objectif**

La **standardisation** consiste à **centrer** les données (soustraction de la moyenne) puis à **réduire** (division par l'écart-type).

Elle transforme la variable pour avoir :

- une **moyenne de 0**
- un **écart-type de 1**

$$z = \frac{x - \mu}{\sigma}$$

avec : x : valeur brute, μ : moyenne et σ : écart-type

Quand l'utiliser ?

- Lorsque les données suivent une distribution **approximativement normale**.
- Lorsque les **modèles sont sensibles à la variance**, comme : **SVM**, **Régression linéaire**, **Réseaux de neurones**, **K-means**

Avantages

- Très utilisé dans la majorité des algorithmes.
- Robuste pour des données symétriques.

limites

- Sensible aux **valeurs extrêmes (outliers)** qui perturbent la moyenne et l'écart-type.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) # X peut être un DataFrame ou array NumPy
```

Min-Max scaling (mise à l'échelle [0,1])

Standardisation (centrage-réduction) :

- **Définition et objectif**

Cette méthode **transforme chaque valeur** pour qu'elle se situe entre 0 et 1, en **préservant les proportions**.

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

- **Quand l'utiliser ?**
- Quand tu veux **préserver la distribution originale** (pas centrer).
- Adaptée à :
 - **KNN**
 - **régression logistique**
 - **réseaux de neurones**
- Idéale pour **images, pixel intensities**, ou pour visualisation.

Avantages

- Chaque valeur est **comprise entre 0 et 1** : pratique pour certains modèles et réseaux de neurones.
- Facile à interpréter visuellement.

Limites

- Très sensible aux **valeurs extrêmes**.
- Ne centre pas les données (moyenne $\neq$ 0).

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```


Transformation logarithmique (log transform)

La transformation logarithmique est l'une des transformations mathématiques les plus couramment utilisées en ingénierie des caractéristiques. Les avantages de la transformation logarithmique sont:

1. Elle permet de traiter des données asymétriques et, après transformation, la distribution devient plus proche d'une distribution normale.
2. Dans la plupart des cas, **l'ordre de grandeur des données change dans la plage des données.**
Par exemple, la différence entre les âges de 15 et 20 n'est pas égale à la différence entre les âges de 65 et 70. En termes d'années, oui, ils sont identiques, mais pour tous les autres aspects, une différence de 5 ans à un jeune âge signifie une différence de magnitude plus élevée. Ce type de données provient d'un processus multiplicatif et la transformation logarithmique normalise les différences de magnitude de cette manière.
3. Elle **diminue également l'effet des valeurs aberrantes** (outliers) en raison de la **normalisation des différences de magnitude**, rendant le modèle plus robuste.

Note importante : Les données sur lesquelles on applique la transformation logarithmique doivent avoir des valeurs strictement positives, sinon une erreur se produira. De plus, vous pouvez ajouter 1 à vos données avant de les transformer. Ainsi, vous vous assurez que la sortie de la transformation sera positive.

Transformations non linéaires: log, racine carrée, Box-Cox

Définition et objectif: Ces transformations servent à :

- Réduire la variabilité extrême
- Corriger l'asymétrie (skewness)
- Rendre les distributions plus proches de la normale

1) Transformation logarithmique

$$x' = \log(x + 1)$$

Quand l'utiliser ?

- Lorsque les données sont **très asymétriques (right-skewed)**.
- Ex : revenus, population, montants

```
import numpy as np
df['revenu_log'] = np.log1p(df['revenu']) # log(1 + x)
```

Transformation racine carrée

$$x' = \sqrt{x}$$

Pour **réduire la variance** dans des données positives avec faibles valeurs extrêmes.

```
df['volume_root'] = np.sqrt(df['volume'])
```

Box-Cox transformation

C'est une **famille paramétrée** de transformations :

$$x' = \frac{x^\lambda - 1}{\lambda} \quad (\lambda \neq 0)$$

$$x' = \log(x) \quad (\lambda = 0)$$

Quand l'utiliser ?

Pour **transformer une distribution vers une forme normale**
Nécessite que les données soient **strictement positives**

```
from scipy.stats import boxcox
```

```
df['revenu_bc'], fitted_lambda = boxcox(df['revenu'] + 1)
```

Transformations non linéaires: log, racine carrée, Box-Cox

Méthode	But principal	Échelle résultat	Sensible aux outliers	Données requises
StandardScaler	Centrage, réduction	Moy 0, std 1	Oui	Toute distribution
MinMaxScaler	Normalisation sur [0, 1]	[0, 1]	Oui	Toute distribution
Log	Réduction des grandes valeurs	Variable	Moins	Positives
Racine carrée	Réduction modérée d'asymétrie	Variable	Moins	Positives
Box-Cox	Approche vers normalité	Variable	Peu	Strictement positives

Laboratoire formatif

Laboratoire formatif

A. Nettoyage et préparation

1. **Charger** le fichier CSV dans un DataFrame.
2. **Supprimer les doublons complets**, s'il y en a.
3. **Uniformiser** les noms de colonnes : minuscules, sans espaces, accents ni caractères spéciaux.
4. **Harmoniser les valeurs** dans la colonne nom : supprimer les espaces, passer en minuscules, enlever les accents.
5. **Corriger les types de données** : convertir âge en **entier** (gérer les chaînes textuelles), convertir salaire en **float** après suppression des symboles monétaires.
6. **Corriger le format** de la colonne date_embauche en format datetime.
7. **Nettoyer** la colonne durée_contrat : convertir toutes les durées en **mois** (ex : "1 an" → 12).
8. **Standardiser** les colonnes numériques : âge, salaire, note_performance avec un **Z-score** (centrage-réduction).

B. Prédiction du départ (classification binaire):

1. **Utiliser l'algorithme KNN et SVM pour prédire les depart des employés**
2. **Comparer les performances** des deux modèles à l'aide de : la **précision** (accuracy), la **matrice de confusion**,